UNIVERSIDADE DE VASSOURAS

CAMPUS SAQUAREMA

MIGRAÇÃO DE ALGORITMOS PARA PYTHON

Curso: Engenharia de Software

Disciplina: Engenharia de Requisitos e Análise de Sistemas

Thiago Pereira Santiago

Eduardo Antony Aguiar Guimarães

Jenifer Gusmão Barbosa

Juan Frota de Oliveira

Juan Rodrigues Laureano de Souza

Mateus Nascimento Sant'anna Luz

Thiago Almeida das Chagas

Wesley Abreu de Aguiar

Saquarema/RJ

2026

1. Introdução

Este trabalho apresenta a migração de quatro programas escritos originalmente em C, Java e COBOL para Python. O objetivo é manter as operações centrais de cada algoritmo e registrar, de maneira simples, seus requisitos, sua estrutura de classes e as tabelas que poderiam ser utilizadas caso os sistemas passassem a armazenar informações em um banco de dados.

O documento “Documento de Requisitos do Sistema – Módulo de Avaliação Acadêmica no SIGA Ensino”, da Universidade Federal de Juiz de Fora (2016), foi utilizado como referência para a organização das seções e a identificação de requisitos funcionais (RF) e não funcionais (NF). O conteúdo funcional da avaliação acadêmica daquele documento não foi transplantado para estes programas.

1.1. Arquivos analisados

Arquivo original	Linguagem	Programa Python
main.c	C	01_funcionarios.py
SistemaAlunos.java	Java	02_alunos_java.py
main.cob	COBOL	03_alunos_cobol.py
SistemaDeVotacao.java	Java	04_votacao.py

1.2. Critérios da migração

- Conservar menus, entradas, saídas e cálculos essenciais presentes em cada arquivo original.
- Empregar classes, listas, dicionários, repetições e condições em nível introdutório de Python.
- Representar nos diagramas as classes realmente criadas nos arquivos Python.
- Distinguir requisitos existentes no algoritmo de pequenas melhorias de validação introduzidas na migração.
- Apresentar os esquemas SQL como uma proposta futura, sem afirmar que os programas já utilizam banco de dados.

1.3. Convenções dos requisitos

RF indica requisito funcional (o que o programa faz), e NF indica requisito não funcional (uma característica de funcionamento). As prioridades foram classificadas como essencial, importante e desejável, de acordo com o padrão do documento de referência. A numeração de RF e NF é reiniciada em cada programa, pois eles são independentes.

2. Sistema de Funcionários (C → Python)

2.1. Descrição geral e origem

O arquivo main.c contém uma estrutura Funcionario com nome, cargo e salário; um vetor que suporta até 100 registros; e um menu para cadastrar, listar, buscar pelo nome e sair. A versão migrada mantém essa lógica com as classes Funcionario e SistemaFuncionarios. Todos os registros permanecem na memória até o encerramento.

Atores: operador do programa. Entradas: nome, cargo, salário e opção do menu. Saídas: confirmação, lista, resultado da busca e avisos.

2.2. Requisitos funcionais

Código	Requisito funcional	Prioridade
RF001	Permitir o cadastro de funcionário com nome, cargo e salário.	Essencial
RF002	Listar os funcionários cadastrados e seus dados.	Essencial
RF003	Buscar um funcionário pelo nome informado.	Essencial
RF004	Limitar o cadastro a 100 funcionários.	Essencial
RF005	Encerrar a execução pelo menu de opção 0.	Essencial

2.3. Requisitos não funcionais

Código	Requisito não funcional	Prioridade
NF001	Executar pelo terminal com Python 3, sem bibliotecas adicionais.	Essencial
NF002	Mostrar mensagens claras ao usuário.	Importante

NF003	Tratar salário não numérico sem fechar a aplicação (melhoria da migração).	Importante

2.4. Especificação dos principais casos de uso

RS001 – Cadastrar funcionário

Ator: Operador. Pré-condição: Há espaço na lista (menos de 100 funcionários).

1. Escolher a opção 1 do menu.
2. Informar nome, cargo e salário.
3. O programa guarda o registro na lista e confirma o cadastro.

Fluxo alternativo / exceção: Se o limite for atingido, o registro não é criado. A versão Python também rejeita valores vazios ou salário inválido.

Pós-condição: A lista passa a conter o novo funcionário.

RS002 – Buscar funcionário

Ator: Operador. Pré-condição: O programa está em execução.

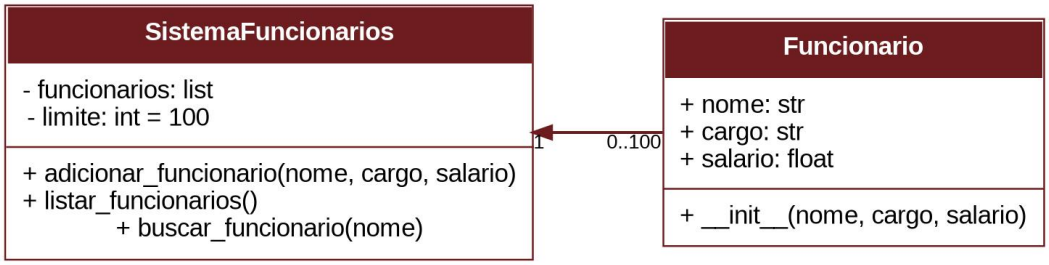
1. Escolher a opção 3.
2. Informar o nome completo.
3. Percorrer a lista e exibir cargo e salário do primeiro nome igual.

Fluxo alternativo / exceção: Quando não existe nome igual, informar “Funcionário não encontrado”.

Pós-condição: Nenhum registro é alterado.

2.5. Diagrama de classes UML

Figura 1 – Classes e relação entre o sistema e os funcionários



Fonte: elaboração própria a partir da estrutura das classes da versão Python.

A classe Funcionario reúne os atributos de cada registro. SistemaFuncionarios armazena a lista, verifica o limite e oferece métodos para cadastrar, listar e buscar. A relação representa que um sistema reúne de zero até cem funcionários.

2.6. Mapeamento de entidades e tabelas

Entidade/tabela proposta	Campos sugeridos e finalidade
Funcionario / funcionarios	id (identificador); nome (texto); cargo (texto); salario (valor numérico).

Uma tabela funcionarios seria suficiente. O campo id é uma proposta de identificação para um futuro banco; não existe no arquivo C de origem. O SQL está em banco_de_dados/01_funcionarios.sql.

2.7. Bancos de dados possíveis

SQLite é a alternativa recomendada para esta atividade, porque trabalha com um único arquivo e pode ser acessado diretamente pelo Python. MySQL ou PostgreSQL seriam opções para um sistema com vários usuários, acesso remoto e maior volume de registros. Nenhuma delas está integrada ao código entregue.

2.8. Exemplos de teste

Cenário de teste	Resultado esperado
Cadastrar Ana, cargo Analista, salário 3200	Cadastro confirmado e registro visível na lista.
Buscar Ana	Exibir cargo e salário.
Buscar nome inexistente	Informar funcionário não encontrado.

Tentar registrar o 101º funcionário	Recusar cadastro por limite atingido.

3. Sistema de Alunos (Java → Python)

3.1. Descrição geral e origem

SistemaAlunos.java utiliza um HashMap para guardar notas por nome e um ArrayList para guardar a ordem dos nomes cadastrados. O menu permite adicionar aluno, listar alunos, buscar nota pelo nome e sair. Na migração, HashMap foi representado por dict e ArrayList por list, dentro da classe SistemaAlunos.

Atores: operador do programa. Entradas: nome do aluno, nota final e opção do menu. Saídas: confirmação, relação de alunos, nota encontrada ou aviso de não encontrado.

3.2. Requisitos funcionais

Código	Requisito funcional	Prioridade
RF001	Cadastrar nome do aluno e nota final.	Essencial
RF002	Exibir a lista de nomes com as notas armazenadas.	Essencial
RF003	Consultar nota a partir do nome informado.	Essencial
RF004	Encerrar a aplicação pela opção 0.	Essencial

3.3. Requisitos não funcionais

Código	Requisito não funcional	Prioridade
NF001	Executar com Python 3 no terminal.	Essencial
NF002	Utilizar dicionário e lista, de forma equivalente às	Importante

	coleções Java.	
NF003	Informar erro ao digitar nota não numérica (melhoria da migração).	Importante

3.4. Especificação dos principais casos de uso

RS001 – Cadastrar aluno

Ator: Operador. Pré-condição: O sistema está aberto.

1. Escolher a opção 1.
2. Informar nome do aluno e nota final.
3. Guardar ou atualizar a nota no dicionário e acrescentar o nome à lista.

Fluxo alternativo / exceção: A migração rejeita nome vazio e nota não numérica. Se o nome for repetido, a nota existente no dicionário é atualizada, mas o nome aparece novamente na lista, como no Java.

Pós-condição: A nota pode ser consultada pelo nome.

RS002 – Buscar nota

Ator: Operador. Pré-condição: O programa está em execução.

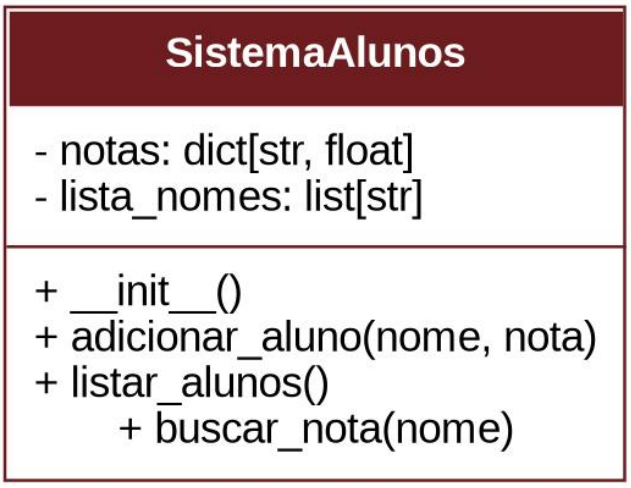
1. Escolher a opção 3.
2. Informar um nome.
3. Consultar o dicionário e mostrar a nota.

Fluxo alternativo / exceção: Quando não encontrar o nome, apresentar mensagem “Aluno não encontrado”.

Pós-condição: Nenhuma informação é alterada.

3.5. Diagrama de classes UML

Figura 2 – Classe SistemaAlunos



Fonte: elaboração própria a partir da estrutura das classes da versão Python.

A única classe do programa Python guarda duas estruturas internas e os métodos de cadastro, listagem e busca. Essa escolha reproduz de forma simples a classe principal do código Java original. A entidade de banco de dados Aluno, descrita a seguir, é uma proposta de persistência, não uma classe adicional do programa.

3.6. Mapeamento de entidades e tabelas

Entidade/tabela proposta	Campos sugeridos e finalidade
Aluno / alunos	id (identificador); nome (texto); nota_final (valor numérico).

A tabela alunos permite identificar cada registro por id, evitando depender de um nome como chave. Essa modelagem de banco é uma melhoria sugerida: no algoritmo original, nomes iguais compartilham a mesma entrada no mapa de notas. O arquivo SQL correspondente é banco_de_dados/02_alunos_java.sql.

3.7. Bancos de dados possíveis

SQLite atende ao trabalho escolar por ser simples. MySQL também é possível caso vários usuários devam consultar as notas. PostgreSQL poderia ser utilizado se o sistema evoluísse para um sistema acadêmico maior. A versão Python entregue ainda trabalha apenas com dados em memória.

3.8. Exemplos de teste

Cenário de teste	Resultado esperado
Cadastrar Beatriz, nota 8,5	Cadastro realizado e nota salva no dicionário.
Buscar Beatriz	Exibir nota 8,5.
Buscar pessoa inexistente	Exibir aviso de aluno não encontrado.
Cadastrar Beatriz novamente com nota 9,0	A nota passa a ser 9,0; o nome aparece duas vezes na lista, como na origem.

4. Cadastro de Alunos e Média (COBOL → Python)

4.1. Descrição geral e origem

O arquivo main.cob define NUM-ALUNOS com valor 5, recebe exatamente cinco nomes e cinco notas, soma as notas e executa a rotina CALCULA-MEDIA. Em seguida, mostra os cinco registros e a média calculada. Não existe um menu de opções nesse algoritmo; a versão Python preserva essa sequência.

Atores: operador do programa. Entradas: cinco pares (nome, nota). Saídas: lista com notas e média aritmética da turma.

4.2. Requisitos funcionais

Código	Requisito funcional	Prioridade
RF001	Solicitar o cadastro de exatamente cinco alunos.	Essencial
RF002	Receber o nome e a nota de cada aluno.	Essencial
RF003	Somar as notas e calcular a média aritmética.	Essencial
RF004	Exibir todos os alunos cadastrados, suas notas e a média final.	Essencial

4.3. Requisitos não funcionais

Código	Requisito não funcional	Prioridade
NF001	Executar pelo terminal com Python 3.	Essencial
NF002	Apresentar as notas e a média com duas casas decimais.	Importante
NF003	Validar nome e nota digitados sem encerrar o programa (melhoria).	Importante

4.4. Regras e cálculo

A média é calculada por (nota 1 + nota 2 + nota 3 + nota 4 + nota 5) ÷ 5. O COBOL utiliza PIC 99V99 nas notas, permitindo dois algarismos antes e dois depois da vírgula; na migração foi mantida a faixa de 0 a 99,99 para as notas. Isso não significa que uma disciplina escolar deva usar notas maiores que 10: o limite aqui foi definido pelo formato do dado na origem.

No COBOL, TOTAL-NOTAS também foi declarado como PIC 99V99, o que pode ser insuficiente para somar cinco notas altas. Em Python a soma utiliza um número sem essa limitação. Essa diferença evita perda de informação e foi registrada como uma melhoria técnica.

4.5. Especificação de caso de uso

RS001 – Cadastrar cinco alunos e calcular média

Ator: Operador. Pré-condição: O programa é iniciado.

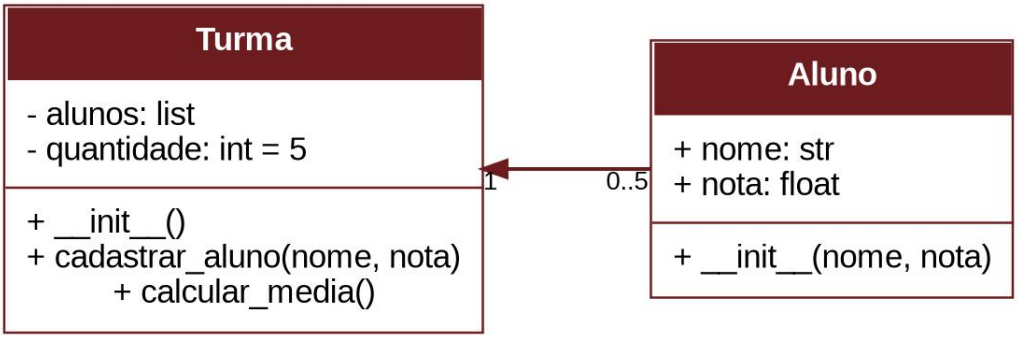
1. Para cada posição de 1 a 5, solicitar o nome e a nota.
2. Guardar cada aluno na lista da turma.
3. Somar as cinco notas, dividir por cinco e mostrar a relação e a média.

Fluxo alternativo / exceção: Caso o nome esteja vazio ou a nota seja inválida, a mesma posição é solicitada novamente; o programa não conta esse registro até haver entrada válida.

Pós-condição: Cinco registros estão disponíveis na memória e a média aparece ao final.

4.6. Diagrama de classes UML

Figura 3 – Turma e seus cinco alunos



Fonte: elaboração própria a partir da estrutura das classes da versão Python.

A classe Aluno representa o nome e a nota. A classe Turma reúne até cinco objetos Aluno e possui o método calcular_media(). No arquivo COBOL original, a mesma informação era organizada em grupos de campos e posições fixas, sem classes.

4.7. Mapeamento de entidades e tabelas

Entidade/tabela proposta	Campos sugeridos e finalidade
Turma / turmas	id (identificador); descricao (texto, proposto para identificar uma turma).
Aluno da turma / alunos_turma	id; turma_id (chave estrangeira para turmas); nome; nota.

A separação em turmas e alunos_turma é uma proposta para permitir mais de uma turma no futuro. O algoritmo original processa apenas um grupo de cinco alunos. O SQL está em banco_de_dados/03_alunos_cobol.sql; a regra de cinco alunos é aplicada pelo código, não por um limite automático do banco.

4.8. Bancos de dados possíveis

SQLite é o banco mais simples para guardar o grupo de alunos em um computador. MySQL e PostgreSQL podem armazenar várias turmas e informações compartilhadas por outros sistemas. Neste exercício, nenhuma persistência foi implementada, pois a origem também não salva arquivos nem registros em banco.

4.9. Exemplos de teste

Cenário de teste	Resultado esperado
Notas 8, 6, 7, 9 e 10	A média mostrada deve ser 8,00.
Digitar nota não numérica	Solicitar nova nota para a mesma posição.
Tentar inserir aluno além do quinto pelo método cadastrar_aluno	Não aceitar o sexto cadastro.

5. Sistema de Votação (Java → Python)

5.1. Descrição geral e origem

SistemaDeVotacao.java contém três opções fixas (“Opção 1”, “Opção 2” e “Opção 3”), um vetor com a quantidade de votos de cada opção e um laço de repetição que termina quando o usuário informa 0. Ao final, a aplicação apresenta os totais. Na migração, a classe SistemaVotacao utiliza duas listas equivalentes.

Atores: pessoa que utiliza o terminal para registrar votos. Entradas: números 1, 2, 3 ou 0. Saídas: confirmação de cada voto, erros de entrada e resultados finais.

5.2. Requisitos funcionais

Código	Requisito funcional	Prioridade
RF001	Exibir três opções de votação numeradas.	Essencial
RF002	Registrar um voto para a opção 1, 2 ou 3.	Essencial
RF003	Recusar escolha fora das opções disponíveis.	Essencial

RF004	Encerrar a votação quando o usuário digitar 0.	Essencial
RF005	Mostrar o total de votos de cada opção após o encerramento.	Essencial

5.3. Requisitos não funcionais

Código	Requisito não funcional	Prioridade
NF001	Executar no terminal com Python 3.	Essencial
NF002	Orientar o usuário quando a entrada não for um número inteiro.	Importante
NF003	Manter os votos apenas durante a execução, como no programa original.	Importante

5.4. Especificação de caso de uso

RS001 – Registrar voto e divulgar totais

Ator: Operador. Pré-condição: O programa está aberto.

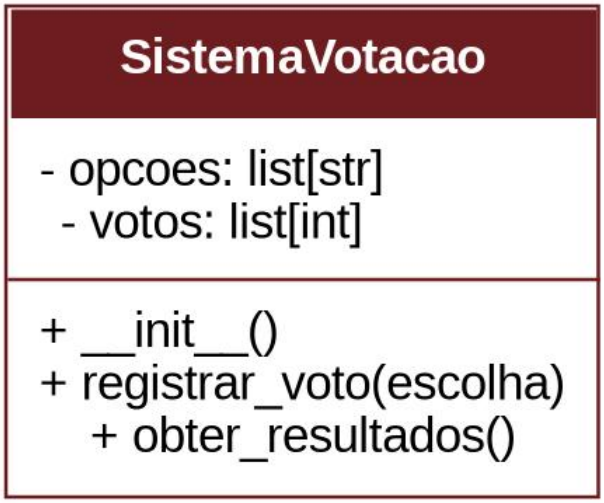
1. Exibir as três opções disponíveis.
2. Receber um número e, se for 1, 2 ou 3, incrementar o contador correspondente.
3. Repetir até receber 0 e então exibir a apuração.

Fluxo alternativo / exceção: Entradas que não são números inteiros ou números fora de 0 a 3 não alteram os totais e geram aviso.

Pós-condição: Os totais são apresentados, sem identificação de eleitores.

5.5. Diagrama de classes UML

Figura 4 – Classe SistemaVotacao



Fonte: elaboração própria a partir da estrutura das classes da versão Python.

A classe contém as opções e os respectivos contadores. O método registrar_voto() acrescenta um voto na posição correta e obter_resultados() devolve a relação final. Não existe cadastro de eleitor nem bloqueio para votos repetidos na origem ou nesta migração.

5.6. Mapeamento de entidades e tabelas

Entidade/tabela proposta	Campos sugeridos e finalidade
OpcaoVotacao / opcoes_votacao	id; descricao (nome da opção); total_votos (quantidade acumulada).

Para reproduzir a funcionalidade atual, basta uma tabela de opções com um contador. Uma tabela separada para registrar votos individuais seria possível em uma versão futura, mas não é exigida nem representada pelo algoritmo fornecido. O SQL de exemplo está em banco_de_dados/04_votacao.sql.

5.7. Bancos de dados possíveis

SQLite é suficiente para uma demonstração local com um operador. Se houvesse múltiplos pontos de votação simultâneos, um banco central como PostgreSQL ou

MySQL seria mais adequado, juntamente com novas regras de controle de concorrência e autenticação, ausentes na aplicação original.

5.8. Exemplos de teste

Cenário de teste	Resultado esperado
Digitar 1, 3, 3 e encerrar com 0	Apuração: opção 1 = 1; opção 2 = 0; opção 3 = 2.
Digitar 7	Mensagem de opção inválida, sem somar voto.
Digitar texto no lugar de número	Mensagem de entrada inválida e nova tentativa.

6. Comparação entre as linguagens e os recursos utilizados

Origem	Estrutura usada na origem	Equivalente Python
C: funcionários	struct + vetor de 100 posições + funções	Classes Funcionario/SistemaFuncionarios + lista + métodos
Java: alunos	HashMap + ArrayList + métodos estáticos	Classe SistemaAlunos + dicionário + lista
COBOL: média	OCCURS 5 TIMES + PERFORM + DIVIDE	Classes Aluno/Turma + laço + soma/divisão
Java: votação	Array de opções + array de contadores + while	Classe SistemaVotacao + listas + while

6.1. Persistência e limitações

Nenhum dos quatro programas de origem contém conexão com banco de dados. Assim, os arquivos SQL de apoio representam possibilidades de evolução, e não funcionalidades já implementadas. Os dados informados desaparecem quando os programas terminam. A aplicação de votação não identifica votantes e não controla

votos duplicados, enquanto o programa de alunos em Java permite efeitos de nomes repetidos no dicionário e na lista.

6.2. Melhorias introduzidas na migração

As versões Python possuem tratamento simples para entrada numérica incorreta e para alguns campos vazios. No cadastro de funcionários, um salário negativo é rejeitado. No cadastro COBOL, a soma e a média deixam de depender de um campo de total com apenas dois algarismos inteiros. Esses ajustes não foram declarados como recursos do código original.

6.3. Como executar e verificar

- Abrir um terminal com Python 3 instalado e entrar na pasta codigos.
- Executar cada arquivo separadamente: python 01_funcionarios.py; python 02_alunos_java.py; python 03_alunos_cobol.py; python 04_votacao.py.
- Para executar os testes automáticos, a partir da pasta principal da entrega, utilizar: executar.py.
- Para verificar modelos de tabelas, executar os arquivos .sql em um banco SQLite; isso não altera os programas Python.

7. Conclusão

A migração dos quatro programas permitiu comparar recursos semelhantes em linguagens diferentes, como vetores, listas, mapas, estruturas, classes e laços de repetição. Python possibilitou expressar essas operações de modo direto e manter as principais regras observadas nos arquivos originais. Os requisitos descritos, os diagramas UML e as tabelas sugeridas ajudam a entender a organização de cada sistema e oferecem uma base simples para futuros aprimoramentos.

8. Glossário

Termo	Significado no trabalho
Requisito funcional (RF)	Função ou ação que o sistema precisa realizar.
Requisito não funcional (NF)	Característica de funcionamento, como execução e clareza.

Caso de uso (RS)	Descrição de como o usuário utiliza uma funcionalidade.
Classe UML	Representação dos atributos e métodos de um objeto.
Entidade / tabela	Estrutura proposta para armazenar registros em um banco de dados.
Chave primária	Identificador de cada registro em uma tabela.
Chave estrangeira	Campo que relaciona uma tabela a outra.
SQLite	Banco de dados relacional que pode ficar em um arquivo local.